Taller Integrador: Automatización y Gestión Avanzada de Bases de Datos

Gestión en Bases de Datos

Presentado por:

Laura Valentina Bautista Clavijo

Jhonatan Alberto Beltrán Cardoso

Bogotá D.C.

Universidad ECCI

2026

1. Análisis del proyecto

Nombre del proyecto	Encontrando Huellitas Un sistema inteligente de reagrupación de mascotas extraviadas y encontradas en la ciudad de Bogotá
Problema que resuelve	La baja tasa de reencuentro entre los dueños de mascotas extraviadas y personas, entidades, refugios y veterinarias que los resguardan en la ciudad de Bogotá, esto es debido a la dispersión de información en las redes sociales y la falta de una algoritmo centralizado de coincidencias morfológica y geográficas. Encontrando Huellitas es un software que centraliza los reportes de mascotas perdidas y encontradas bajo un formato estandarizado. Su propuesta de valor es un motor de coincidencias que cruza automáticamente ambos tipos de reporte por especie, raza, color y zona, y conecta a particulares, fundaciones, refugios y veterinarias mediante mensajería interna.
Usuarios principales	Los usuarios identificados y utilizados para el desarrollo de este proyecto son: ● Ciudadanos particulares: Propietarios que reportan mascotas perdidas o ciudadanos que reportan hallazgos temporales de animales en la geografía de la ciudad. ● Entidades reportantes: Fundaciones, refugios y veterinarias registradas que resguardan animales. ● Administrador del sistema: Encargado de la supervisión global, moderación de reportes e intervenciones directas.
Módulos principales	Los módulo centrales identificados para el correcto desarrollo del proyecto son: ● Módulo de autenticación y perfiles: Gestión de sesiones, tipos de roles y perfiles normalizados. ● Módulo de reportes de mascotas: Registro con características morfológicas como la especie, raza, colores, tamaño e identificación por ubicación. ● Motor de coincidencias - matching engine: Evaluación ponderada de atributos morfológicos e interpolación de zonas. ● Módulo de mensajería y contacto: Canal interactivo directo entre usuarios vinculados a coincidencias. ● Módulo de control administrativo: Auditoría de estado, cierre transaccional de casos y moderación de contenido.

2. Funciones almacenadas

   Las funciones escalares en la base de datos permite encapsular los cálculos repetitivos de la lógica de negocio y ser invocados de forma directa dentro de esas sentencias SELECT o cláusulas del tipo WHERE.

Elemento	Función 1	Función 2
Nombre	fn_coincidencia_zona	fn_calcular_similitud_total
Objetivo	Calcular la ponderación geográfica de coincidencia entre dos zonas o localidades de Bogotá.	Evaluar el porcentaje ponderado de compatibilidad morfológica entre un reporte de pérdida y uno de hallazgo.
Parámetros de entrada	zona1 VARCHAR(50), zona2 VARCHAR(50)	p_id_perdida INT, p_id_hallazgo INT
Valor retornado	DECIMAL(3,2) (Retorna 0.20 si las zonas son idénticas, 0.00 si son distintas).	DECIMAL(3,2) (Valor escalar entre 0.00 y 1.00 de compatibilidad total).
Justificación	Permite parametrizar el componente de ubicación que corresponde al 20% del peso en el algoritmo, dentro de las consultas SQL directas sin saturar la aplicación backend.	Centraliza en el SGBD la fórmula del algoritmo (que consiste en la especie estricta, la raza 35%, el color 30%, la zona 20%, el tamaño 15%) optimizando la velocidad de respuesta en búsquedas masivas.

3. Procedimientos almacenados

   Los procedimientos almacenados permiten agrupar secuencias de instrucciones SQL complejas en el motor de base de datos, garantizando el rendimiento, la reutilización y el encapsulamiento de lógica transaccional.

Elemento	Procedimiento 1	Procedimiento 2
Nombre	sp_cerrar_reporte	sp_generar_coincidencias_masivas
Objetivo	Ejecutar de forma transaccional la resolución o el borrado lógico de un reporte de mascota y la limpieza de sus dependencias.	Analizar todo el universo de mascotas activas y registrar en la tabla coincidencia aquellas que igualen o superen el umbral configurado del 50%.
Parámetros	IN p_id_reporte INT, IN p_nuevo_estado ENUM('ENCONTRADO','RESUELTO','CERRADO')	IN p_umbral_minimo DECIMAL(3,2)

Tablas involucradas	reporte_mascota , coincidencia , mensaje_contacto	reporte_mascota , reporte_perdida , reporte_hallazgo , coincidencia
Acción realizada	Desvincula los mensajes de contacto asignados a coincidencias del reporte, elimina las coincidencias activas y actualiza el estado a RESUELTO o CERRADO . En caso de error aplica ROLLBACK .	Recorre los reportes en estado PERDIDO y los contrasta con los reportes ENCONTRADOde la misma especie, insertando únicamente las coincidencias válidas con su respectivo porcentaje.

4. Análisis de transacciones

   Una transacción garantiza que un grupo de operaciones se ejecuten como una unidad indivisible donde se cumple la propiedad ACID:Atomicidad, Consistencia, Aislamiento y Durabilidad.

Proceso crítico	¿Por qué se requiere una transacción?	¿Qué podría ocurrir si falla?
Cierre o eliminación de reporte (sp_cerrar_reporte)	Involucra tres pasos secuenciales dependientes: desvincular los mensajes activos de una coincidencia, eliminar las filas de la tabla coincidencia y cambiar el estado del reporte en reporte_mascota .	Si el sistema elimina las coincidencias pero se interrumpe antes de actualizar el estado de la mascota, el reporte quedaría en estado PERDIDO pero sin la posibilidad de ser contactado por la interfaz, creando inconsistencias de huérfanos en la Base de Datos.
Confirmación de reencuentro entre usuarios	Al confirmar que una mascota fue devuelta, el sistema debe cambiar de forma simultánea el estado del reporte de pérdida a RESUELTO , el del reporte de hallazgo a RESUELTO y actualizar la coincidencia a CONFIRMADA .	Si solo se actualiza el reporte de pérdida y el de hallazgo falla, la mascota encontrada seguiría figurando como disponible en la ciudad, permitiendo que otros usuarios inicien chats sobre un animal ya entregado.

5. Concurrencia
    - Situación de acceso simultáneo en el sistema de “Encontrando Huellitas”
      Dos usuarios sea uno tipo administrador, entidad reportante y un administrador intenta actualizar o cambiar el estado del mismo reporte de mascota al mismo tiempo.
      Dos personas que hallaron mascotas similares intentan iniciar una solicitud de contacto sobre el mismo reporte de pérdida simultáneamente.
    - Problemas que podrían generarse
      Condición de carrera - race condition que corresponde a la pérdida de actualización “lost update”, donde el cambio de un usuarios sobreescribe la modificación hecha milisegundos antes por el administrador.
      Las lecturas sucias o inconsistentes; ya que un usuario envie un mensaje a través de una coincidencia que está siendo eliminada en ese preciso instante por la resolución del caso.
    - Gestión propuesta para evitar inconsistencia
      Bloqueo pesimista “SELECT … FOR UPDATE”, en operaciones críticas como el cierre de reportes, se bloquea temporalmente la fila del reporte durante la transacción para impedir lecturas o escrituras concurrentes hasta completar el COMIT.
      Nivel de aislamiento InnoDB, la configuración del motor MySQL en nivel REPEATABLE READ para asegurar que los datos leídos dentro de una transacción permanezcan constantes a pesar de peticiones simultáneas.

6. Partición de datos
    - Tabla con crecimiento crítico del tiempo: mensaje_contacto y coincidencia
      Donde con el uso continuo en Bogotá, la tabla mensaje_contacto acumulará miles de interacciones de chat, mientras que reporte_mascota crecerá constantemente. La tabla seleccionada para particionamiento es la de reporte_mascota.
    - Tipo de partición propuesta: Se propone una partición por rango basado en la fecha “RANGE BY YEAR” combinado con la partición de listas o zonas “LIST BY REgION”
    - Justificación técnica: La búsqueda de mascotas extraviadas se concentra prioritariamente en reportes recientes como el año en curso, al particionar por

año “YEAR(fecha_reporte)”, el motor de la base de datos ejecuta un partition pruning, omitiendo años pasados y acelerando las consultas del matching engine en más de un 70%. En cambios los reportes antiguos o históricos pueden archivarse en particiones frías en discos secundarios sin afectar la velocidad operativa del sistema actua

l

7. Diseño integrado - esquema arquitectónico del sistema

   Flujo de integración de capas del SGBD en el sistema Encontrando Huellitas

   [Capa del cliente e interfaz WEB]

   - Petición de usuario donde se registra el reporte, se envían mensajes y se cierran los casos de las mascotas (Registrar Reporte / Enviar Mensaje / Cerrar Caso).

   [API BACKEND - Node.js / Express]

   - Validación de credenciales hash (bcrypt) y coordina la llamada a la base de datos

     [MOTOR SGBD - MySQL / InnoDB]

     - Manejador de concurrencia: Se aplica bloqueo pesimista “FOR UPDATE” e instalación transaccional.
     - Procedimiento almacenada “sp_cerrar_reporte”: Se inicia la transacción “START TRANSACTION”
     - Función escalar “fn_coincidencia_zona” En esta se evalúa en memoria la ponderación de localidad.
     - Triggers Automáticos “trg_auditoria_estado_reporte”: En este se registra el cambio en auditoria_reporte .
     - Gestor de Almacenamiento: Consulta rápida mediante el partition pruning en reporte_mascota.

     [Confirmación o revertido]

     - En éxito: COMMIT con los datos consolidados
     - En fallo: ROLLBACK sin estado parcial corrompido

8. Reflexión final y argumentación técnica
   - Concepto más útil para el proyecto: Los stored procedures transaccionales junto con los triggers de auditoría.; puesto que "Encontrando Huellitas"

maneja estados vinculados entre dos partes de quien pierde y quién encuentra, garantizando que la desvinculación de chats y la actualización del estado ocurran de forma atómica evita que el sistema pierda consistencia o deje datos huérfanos.

- Proceso más beneficiado por la automatización: El cierre transaccional de reportes y la audición de cambios, donde al momento de mover esta lógica al motor MySQL mediante sp_cerrar_reporte y trg_auditoria_estado_reporte , se elimina el riesgo de que errores de sintaxis o fallos de red en el backend de la herramienta Node.js dejen datos parcialmente modificados.
- Riesgos de una deficiente gestión transaccional: La corrupción de la integridad referencial, las mascotas marcadas como encontradas que siguen apareciendo en búsquedas, la pérdida de historial de auditoría frente a la moderación administrativa y estados inconsistentes entre la interfaz y la base de datos.
- Contribución de la partición de datos: Maximiza el rendimiento I/O disminuyendo el tiempo de respuesta de las consultas complejas de coincidencia al evitar escaneos secuenciales completos del Full Table Scans sobre los registros históricos de años anteriores